

Documento explicativo del proyecto final

Generador de Recetas con Inventario | Tecnologías Web

Integrantes: Sean Paul Marquez Toro, Reyner David Barbosa de la Rosa y Ruben Andres Corro Blanco

Resumen ejecutivo

Se desarrollo y despliego una aplicacion web para registrar ingredientes disponibles en casa y obtener recetas generadas o recomendadas a partir del inventario. El proyecto cumple el stack solicitado: Python con FastAPI, PostgreSQL, integracion configurable con OpenRouter como proveedor LLM, Docker Compose, VPS en AWS Lightsail, dominio propio y SSL.

Entregable	Resultado final
Repositorio	https://github.com/rubencorrob-sudo/generador-recetas-inventario
Aplicacion	https://www.recetasruben.xyz
Swagger	https://www.recetasruben.xyz/docs
PDF tecnico	docs/informe-proyecto-recetas.pdf
Respaldo temporal	https://54-236-36-56.sslip.io

Que se construyo

- Backend modular en FastAPI con routers para autenticacion, ingredientes, recetas, recomendaciones y calificaciones.
- Base de datos PostgreSQL en produccion con modelos SQLAlchemy para usuarios, ingredientes, recetas y ratings.
- Interfaz web tipo dashboard con registro/login, inventario, preferencias culinarias, recomendaciones y recetas guardadas.
- Servicio LLM encapsulado en app/services/llm_service.py, preparado para OpenRouter y modelo openai/gpt-4o-mini.
- Motor local de recomendaciones con productos de canasta familiar, compatibilidad, faltantes y desglose de ingredientes.
- Documentacion automatica OpenAPI/Swagger disponible en /docs.

Mejoras realizadas durante el desarrollo

- Se amplio el catalogo de productos comunes de canasta familiar para mejorar las recomendaciones.
- Se agrego desglose de ingredientes por receta: disponibles, faltantes, opcionales y basicos asumidos.
- Se enriquecio la experiencia visual con imagenes, favicon, tarjetas de recetas y un dashboard mas presentable.
- Se agregaron favoritos, calificaciones, busqueda, historial y copia de recetas al portapapeles.
- Se mejoro el PDF de entrega con arquitectura, diagrama ER, capturas y enlaces clicables.
- Se actualizaron README y documentos de arquitectura con URLs reales y datos del grupo.

Despliegue y dominio

La aplicacion se despliego en una VPS Ubuntu de AWS Lightsail con IP publica 54.236.36.56. Se uso Docker Compose para levantar tres servicios principales: app, db y caddy. Caddy funciona como proxy inverso y emite certificados SSL automaticos con Let's Encrypt.

1. Se clono el repositorio en la VPS y se configuraron variables de entorno en .env.
2. Se levanto PostgreSQL, FastAPI/Uvicorn y Caddy con Docker Compose.
3. Se abrieron puertos 80 y 443 en Lightsail para HTTP/HTTPS.
4. Primero se uso DuckDNS, pero algunas redes lo bloqueaban por ser dynamic DNS.
5. Se uso sslip.io como respaldo tecnico mientras se resolvia el dominio propio.
6. Finalmente se compro recetasruben.xyz en Namecheap y se configuraron registros A para @ y www hacia 54.236.36.56.
7. Caddy emitio SSL de Let's Encrypt para recetasruben.xyz y www.recetasruben.xyz.

Configuracion DNS final

Tipo	Host	Valor	TTL
A Record	@	54.236.36.56	Automatic
A Record	www	54.236.36.56	Automatic

Validaciones realizadas

- La aplicacion en produccion respondio HTTP 200 en <https://www.recetasruben.xyz>.
- Swagger respondio HTTP 200 en <https://www.recetasruben.xyz/docs>.
- El dominio sin www tambien respondio correctamente en <https://recetasruben.xyz>.
- El certificado SSL fue emitido por Let's Encrypt y validado por Caddy.
- La base de datos PostgreSQL quedo healthy en Docker Compose.
- Los tests locales del proyecto pasaron: 14 passed.
- Los cambios de PDF, README, Caddyfile y documentos fueron subidos a GitHub.

Como se debe sustentar

- Explicar que el usuario registra ingredientes y el sistema recomienda o genera recetas segun el inventario.
- Mostrar el flujo: registro/login, agregar ingredientes, ver recomendaciones, generar receta y guardar/calificar.
- Mostrar Swagger para evidenciar endpoints REST y documentacion automatica.
- Explicar que OpenRouter esta encapsulado en un servicio interno y no se expone como chatbot libre.
- Explicar la arquitectura Docker: Caddy SSL -> FastAPI -> PostgreSQL, con OpenRouter como proveedor externo.
- Mostrar el PDF tecnico con arquitectura, diagrama ER y capturas.

Checklist final para enviar

Elemento	Estado	Evidencia
Repositorio publico	Listo	GitHub con commits y documentos actualizados
Aplicacion en produccion	Listo	https://www.recetasruben.xyz responde 200
SSL	Listo	Certificado Let's Encrypt emitido por Caddy
Swagger	Listo	https://www.recetasruben.xyz/docs responde 200
PDF tecnico	Listo	Incluye arquitectura, ER, capturas y enlaces
Documento explicativo	Listo	Este documento resume desarrollo y despliegue

Evidencias tecnicas rapidas

- Docker Compose quedo con servicios app, db y caddy levantados.
- La base de datos quedo healthy en el contenedor de PostgreSQL.
- Caddy sirve HTTPS para recetasruben.xyz y www.recetasruben.xyz.
- El PDF tecnico y este documento quedaron dentro de la carpeta docs/.
- El repositorio remoto quedo actualizado en la rama main.

Notas finales

El repositorio no contiene credenciales reales. Las claves sensibles se manejan por variables de entorno. El dominio propio y SSL ya cumplen el requisito del parcial. La URL de respaldo solo se conserva como plan B tecnico, pero la URL principal para entrega es el dominio propio recetasruben.xyz.